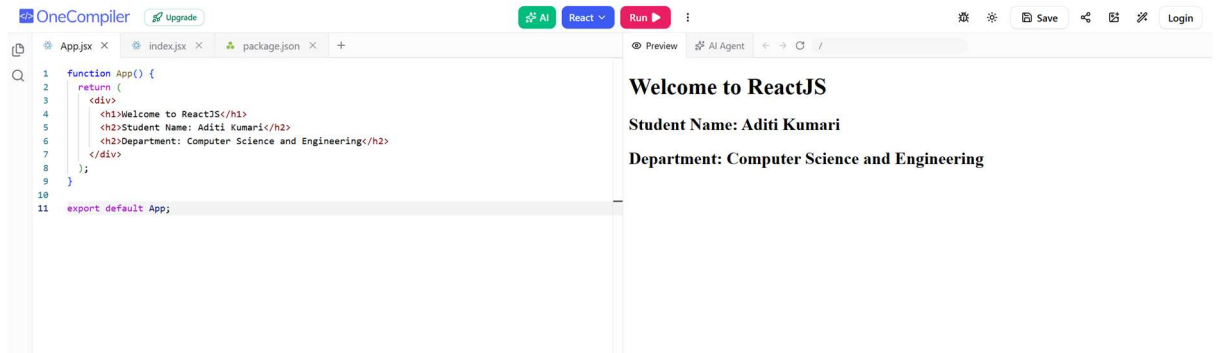


ADITI KUMARI

2024053487

Assignment :- 4

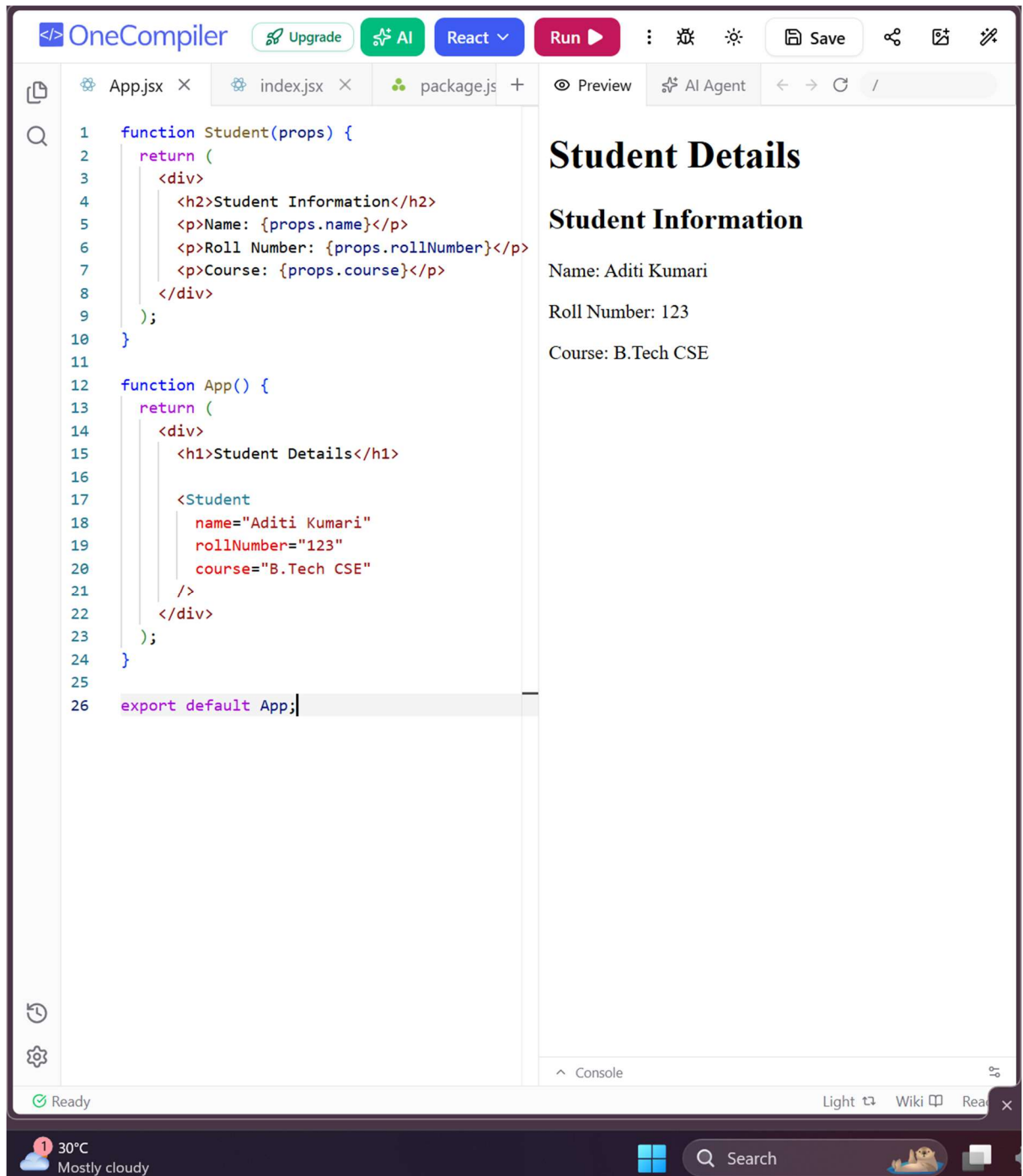
1. Develop a ReactJS application that displays a welcome message, student name, and department using functional components and JSX.



```
1 function App() {  
2   return (  
3     <div>  
4       <h1>Welcome to ReactJS</h1>  
5       <h2>Student Name: Aditi Kumari</h2>  
6       <h2>Department: Computer Science and Engineering</h2>  
7     </div>  
8   );  
9 }  
10  
11 export default App;
```

Welcome to ReactJS
Student Name: Aditi Kumari
Department: Computer Science and Engineering

2. Create a ReactJS application using reusable components to display student information. Pass student name, roll number, and course as props from a parent component to a child component.



3. Develop a ReactJS counter application using the useState hook. Implement Increment, Decrement, and Reset buttons to update the counter value.

The image shows a web-based IDE called OneCompiler. The top bar includes a logo, an 'Upgrade' button, an 'AI' button, a 'React' dropdown, a 'Run' button, and icons for settings, search, and save. Below the top bar, there are tabs for 'App.jsx', 'index.jsx', and 'package.json'. The main editor area on the left displays the following code:

```
1 import { useState } from 'react';
2
3 function App() {
4   const [count, setCount] = useState(0);
5
6   return (
7     <div>
8       <h1>Counter: {count}</h1>
9
10      <button onClick={() => setCount(count + 1)}>
11        Increment
12      </button>
13
14      <button onClick={() => setCount(count - 1)}>
15        Decrement
16      </button>
17
18      <button onClick={() => setCount(0)}>
19        Reset
20      </button>
21    </div>
22  );
23 }
24
25 export default App;
```

On the right side, there is a 'Preview' tab showing the rendered output of the code. It displays a large heading 'Counter: 1' and three buttons: 'Increment', 'Decrement', and 'Reset'.

4. Create a ReactJS form that accepts a student's name and email. Handle input changes and display the submitted information using state management and event handling.

OneCompiler

Upgrade

AI

React

Run

Save

AI Agent

App.jsxindex.jsxpackage.json

1import { useState } from 'react';

2

3function App() {

4 const [name, setName] = useState('');

5 const [email, setEmail] = useState('');

6 const [submittedName, setSubmittedName] = useState('');

7 const [submittedEmail, setSubmittedEmail] = useState('');

8

9 function handleSubmit(event) {

10 event.preventDefault();

11

12 setSubmittedName(name);

13 setSubmittedEmail(email);

14 }

15

16 return (

17 <div>

18 <h1>Student Form</h1>

19

20 <form onSubmit={handleSubmit}>

21 <label>Name: </label>

22 <input

23 type="text"

24 value={name}

25 onChange={(event) => setName(event.target.value)}

26 </input>

27

28

29

30 <label>Email: </label>

31 <input

32 type="email"

33 value={email}

34 onChange={(event) => setEmail(event.target.value)}

35 </input>

36

37

38

39 <button type="submit">Submit</button>

40 </form>

41

42 <h2>Submitted Information</h2>

Student Form

Name: Aditi Kumari

Email: akumari@student.gitam.edu

Submit

Submitted Information

Name: Aditi Kumari

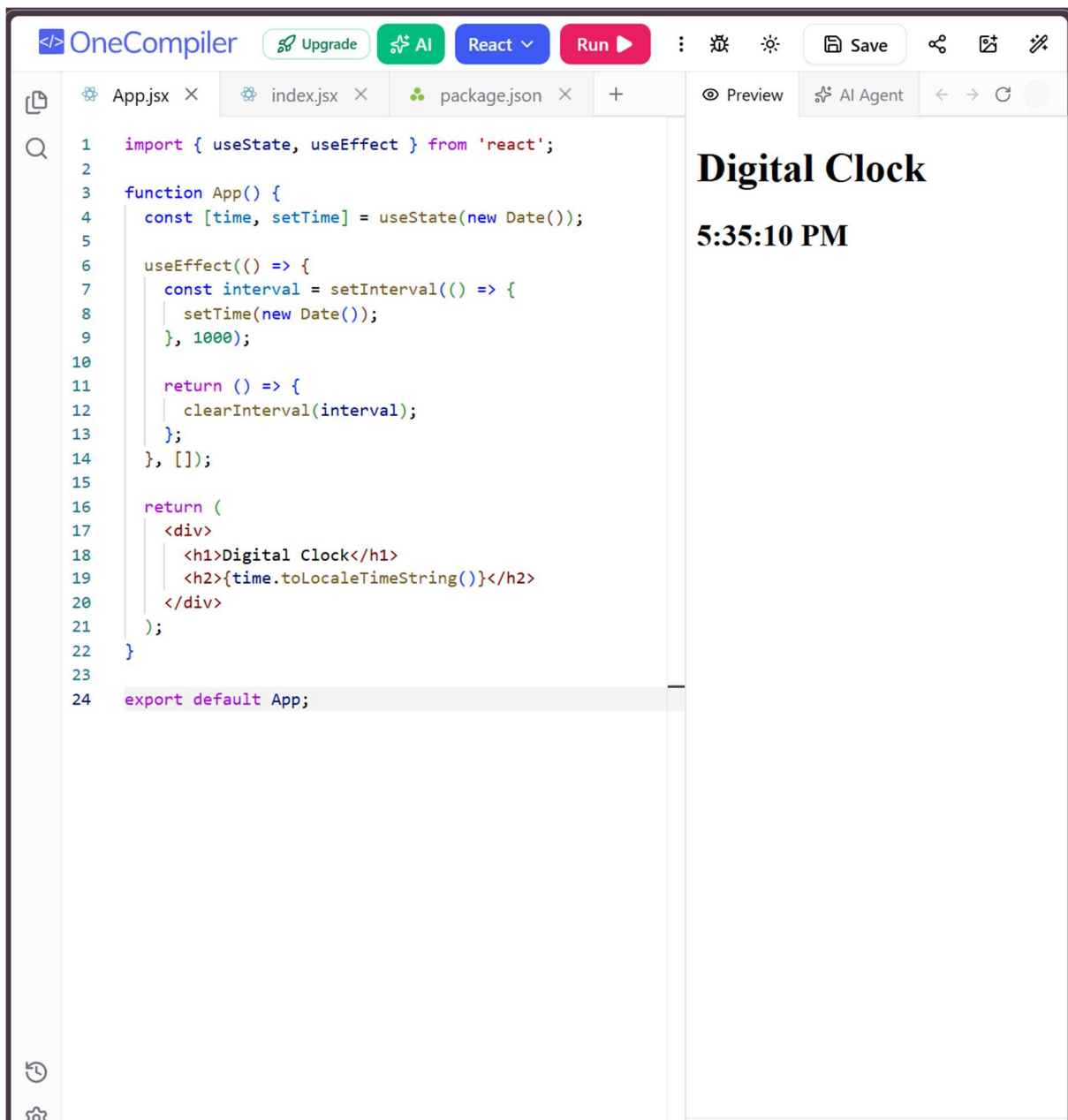
Email: akumari@student.gitam.edu

Console

Readv

light Wiki Real

5. Develop a ReactJS digital clock using the `useEffect` hook that updates every second. Implement proper cleanup of the interval when the component is unmounted.



6. Create a ReactJS application that fetches and displays a list of users from a public API using `useEffect`. Handle loading and error states.

```
function App() {  
  useEffect(() => {  
    fetch('https://jsonplaceholder.typicode.com/users')  
      .then((response) => {  
        if (!response.ok) {  
          throw new Error('Failed to fetch users');  
        }  
        return response.json();  
      })  
      .then((data) => {  
        setUsers(data);  
        setLoading(false);  
      })  
      .catch((error) => {  
        setError(error.message);  
        setLoading(false);  
      });  
  }, []);  
  
  if (loading) {  
    return <h2>Loading...</h2>;  
  }  
  
  if (error) {  
    return <h2>Error: {error}</h2>;  
  }  
  
  return (  
    <div>  
      <h1>User List</h1>  
      {users.map((user) => (  
        <p key={user.id}>  
          {user.name} - {user.email}</p>  
      ))}  
    </div>  
  );  
}
```

User List

Leanne Graham - Sincere@april.biz

Ervin Howell - Shanna@melissa.tv

Clementine Bauch - Nathan@yesenia.net

Patricia Lebsack - Julianne.OConner@kory.org

Chelsey Dietrich - Lucio_Hettinger@annie.ca

Mrs. Dennis Schulist - Karley_Dach@jasper.info

Kurtis Weissnat - Telly.Hoeger@billy.biz

Nicholas Runolfsdottir V - Sherwood@rosamond.me

Glenna Reichert - Chaim_McDermott@dana.io

Clementina DuBuque - Rey.Padberg@karina.biz

7. Develop a ReactJS application using the Context API to implement Light and Dark themes. Allow users to switch themes across multiple components without prop drilling.

```
5 function ThemeProvider({ children }) {
10   {children}
11   </ThemeContext.Provider>
12 };
13 }
14
15 function ThemeButton() {
16   const { dark, setDark } = useContext(ThemeContext);
17
18   return (
19     <button onClick={() => setDark(!dark)}>
20       Switch to {dark ? 'Light' : 'Dark'} Theme
21     </button>
22   );
23 }
24
25 function Content() {
26   const { dark } = useContext(ThemeContext);
27
28   return (
29     <div
30       style={{
31         backgroundColor: dark ? 'black' : 'white',
32         color: dark ? 'white' : 'black',
33         padding: '30px'
34       }}
35     >
36       <h1>{dark ? 'Dark Theme' : 'Light Theme'}</h1>
37       <ThemeButton />
38     </div>
39   );
40 }
41
42 function App() {
43   return (
44     <ThemeProvider>
45       <Content />
46     </ThemeProvider>
47   );
48 }
49
50 export default App;
```

Dark Theme

Switch to Light Theme

8. Create a ReactJS shopping cart application using functional components, `useState`, and props. Implement Add to Cart and Remove from Cart functionality.

The screenshot displays the OneCompiler IDE with a ReactJS application. The code editor on the left shows the following code:

```
15 function Cart({ cart, removeFromCart }) {
16   // ...
17 }
18
19 function App() {
20   const [cart, setCart] = useState([]);
21
22   function addToCart(product) {
23     setCart([...cart, product]);
24   }
25
26   function removeFromCart(index) {
27     setCart(cart.filter((item, i) => i !== index));
28   }
29
30   return (
31     <div>
32       <h1>Shopping Cart</h1>
33
34       <Product
35         name="Laptop"
36         price={50000}
37         addToCart={addToCart}
38       />
39
40       <Product
41         name="Headphones"
42         price={2000}
43         addToCart={addToCart}
44       />
45
46       <Cart
47         cart={cart}
48         removeFromCart={removeFromCart}
49       />
50     </div>
51   );
52 }
53
54 export default App;
```

The preview on the right shows the rendered application:

Shopping Cart

Laptop
Price: ₹50000
Add to Cart

Headphones
Price: ₹2000
Add to Cart

Shopping Cart

Laptop - ₹50000
Remove

Laptop - ₹50000
Remove

Headphones - ₹2000
Remove

9. Develop a ReactJS student attendance management application using Context API and `useReducer`. Implement actions to mark and display student attendance.

The screenshot shows the OneCompiler IDE with a ReactJS application for "Student Attendance". The code is as follows:

```
25 function AttendanceProvider({ children }) {
33 }
34
35 function Attendance() {
36   const { students, dispatch } = useContext(AttendanceContext);
37
38   return (
39     <div>
40       <h1>Student Attendance</h1>
41
42       {students.map((student) => (
43         <div key={student.id}>
44           <p>
45             {student.name} -{' '}
46             {student.present ? 'Present' : 'Absent'}
47           </p>
48
49           <button
50             onClick={() =>
51               dispatch({
52                 type: 'MARK_PRESENT',
53                 id: student.id
54               })
55             }
56           >
57             Mark Present
58           </button>
59         </div>
60       ))}
61     </div>
62   );
63 }
64
65 function App() {
66   return (
67     <AttendanceProvider>
68       <Attendance />
69     </AttendanceProvider>
70   );
71 }
72
73 export default App;
```

The preview on the right shows the rendered application:

Student Attendance

Aditi - Present

Mark Present

Rahul - Present

Mark Present

Priya - Absent

Mark Present

10. Create a ReactJS task management application using reusable components, `useState`, `useEffect`, and `Context` API. Implement adding and deleting tasks and sharing task data across components.

OneCompiler

UpgradeAIReactRun

App.jsxindex.jsxpackage.json

PreviewAI Agent

32function AddTask() {
49 value={task}
50 onChange={(event) => setTask(event.target.value)}
51 />
52
53 <button type="submit">Add Task</button>
54 </form>
55 };
56 }
57
58 function TaskList() {
59 const { tasks, deleteTask } = useContext(TaskContext);
60
61 return (
62 <div>
63 <h2>Tasks</h2>
64
65 {tasks.map((task, index) => (
66 <div key={index}>
67 <p>
68 {task}
69 <button onClick={() => deleteTask(index)}>
70 Delete
71 </button>
72 </p>
73 </div>
74)>
75 </div>
76)>
77 }
78
79 function App() {
80 return (
81 <TaskProvider>
82 <h1>Task Management</h1>
83 <AddTask />
84 <TaskList />
85 </TaskProvider>
86)>
87 }
88
89 export default App;

Task Management

Add Task

Tasks

Lab Observation Delete

Exam Delete

Console (3)